

Workhorse Image Scaler

This runbook covers operations of Workhorse's built-in image scaler.

Workhorse cooperates with Rails to route and handle image scaling requests. Workhorse itself is responsible for proxying traffic, serving image data from either remote or block storage, and handling conditional GETs. Image scaling itself, however, is not handled by Workhorse, but by a companion tool called `gitlab-resize-image` (just “scaler” hereafter) that Workhorse shells out to for every such request.

Image scaler requests are ordinary web requests to images served via the `/uploads/` path and which furthermore carry a `width` parameter, e.g.:

- `/uploads/-/system/group/avatar/22/avatar_w300.png?width=16`
- `/uploads/-/system/user/avatar/1/avatar.png?width=64`

NOTE:

- we currently only rescale project, group and user avatars
- we only rescale PNGs and JPEGs (see `SAFE_IMAGE_FOR_SCALING_EXT`)
- we only rescale images when requesting a width defined by `ALLOWED_IMAGE_SCALER_WIDTHS`
- we only rescale images that do not exceed a configured size in bytes (see `max_filesize`)
- we only rescale images if enough scaler processes are available (see `max_scaler_procs`)

NOTE:

If you are confident it is the scaler itself failing, and not an ancillary system such as GCS (where images are stored), you can quickly restore imaging functionality by toggling the `dynamic_image_resizing` feature flag off. This is an `ops` type toggle and should continue to exist even after this feature is fully rolled out.

Dashboards & Logs

In case of an incident, look at:

1. The `imagescaler` component panels in the web overview dashboard for general component health and performance
2. Thanos total requests metric for a more direct breakdown of scaler `status` per environment etc.
3. Kibana logs for detailed request logs
4. Kibana error breakdown counting scaler errors by message.

Failure modes

Generally, three outcomes are possible:

- we scaled then served the rescaled image (good; this is a 200)

- we failed running the scaler, but served the original (bad; this is still a 200)
- we failed to serve anything, rescaled or otherwise (worse; this is a 500)

The following sections describe how we might run into the last two types of failures.

Scaler failed, original was served

This can happen in two cases, outlined below. In both cases, we fail over to serving the original (usually much larger) image; this ensures we do not break functionality, but comes with client-side performance drag and higher egress traffic.

There are more scaling requests than available scalers Context:

We currently cap the number of scalers that may execute concurrently via Workhorse's `max_scaler_procs` config field. If we trip that threshold, we will start ignoring new scaler requests.

Symptoms:

- Users may notice that image downloads take longer than usual
- The saturation metric on this panel will be degraded.
- In Kibana, you will see error logs saying `too many running scaler processes (x / y) (with x > y)`
- In Thanos, `gitlab_workhorse_image_resize_requests_total` will have an elevated rate of `status="served-original"`

To understand where the extra traffic might originate from, look for request spikes on these dashboards:

- User avatars
- Project avatars
- Group avatars

Actions:

We should assess whether we are merely dealing with a short burst of additional requests or if we should consider raising the ceiling for `max_scaler_procs`. It might help to get a process listing from affected nodes and:

- See if scaler processes are getting stuck by looking at process listings (look for `gitlab-resize-image` procs). To unclog the pipes, killing these processes might be the easiest remedy.
- See if scaler processes are finishing, but take a long time to complete (anything above a few dozen to a hundred milliseconds is too slow). The most likely explanation is that either the node is CPU starved (image scaling is a CPU bound operation) or that writing the scaled image back

out to the client is taking a long time due to slow connection speed or other network bottlenecks.

The `max_scaler_procs` setting is set in Workhorse's `config.toml`. For example:

```
[image_resizer]
  max_scaler_procs = XX
```

Note that we can always scale out to reduce pressure on this by running more Workhorse nodes.

The scaler process did not start Context:

This means that we could not fork into `gitlab-resize-image`.

Symptoms:

- Users may notice that image downloads take longer than usual
- In Kibana, you will see error logs saying `fork into scaler process: <reason>`
- In Thanos, `gitlab_workhorse_image_resize_requests_total` will have an elevated rate of `status="served-original"`

Actions:

This is unlikely to “just happen” and your best bet is to look at logs in Kibana to understand why.

No image was served

Context:

This means we were entirely unable to serve an image to the client. This will always result in a 500, and is a user facing error. Unfortunately, there are also countless reasons for why this might happen.

Symptoms:

- Users may see broken images or no images at all
- In Thanos, `gitlab_workhorse_image_resize_requests_total` will have an elevated rate of `status="request-failed"`
- In Kibana, request logs for the scaler will contain error messages

Actions:

This is likely to be highly contextual, but a few things to look out for:

- Are we failing to serve any data? This could be indicated by the `json.written_bytes` field in Kibana logs being 0
- Did we previously fail over to the original image and did we fail to serve that, or did we scale successfully but failed to serve the rescaled image?

Scaler failure should be indicated by additional error logs preceeding the serving failure.

- Was there a problem accessing the image in object storage? Check if there is a problem with GCS credentials.
- Did `gitlab-resize-image` return with a non-zero exit code? Scan logs for why this happened.
- Are clients timing out and closing the connection? This could indicate that we are taking too long to serve image data.